

Rapport de Stage de Fin d'Études

Pour l'obtention du Diplôme d'Ingénieur d'État et du Master

Conception et mise en œuvre d'un pipeline automatisé de contrôle de la qualité des données : Cas des données CSP et KYC

Filière :

Réalisé par :

SAAD BENDAHOU

MIAGE (Méthodes Informatiques
Appliquées à la Gestion des
Entreprises)
MIAGE I2A (Informatique et
Intelligence Artificielle)

Encadrant de l'École :

Mr. Marouan Skiti

Encadrants de CIH BANK :

Mr. Mohamed Agouzzal

Mr. Marouan Naoui

Année Universitaire : 2025 – 2026

Dédicaces

Je dédie ce modeste travail :

À mes chers parents,

Aucun mot, aucune dédicace ne saurait exprimer mon amour, mon respect et ma profonde gratitude pour vos sacrifices, votre soutien inconditionnel et vos prières qui m'ont toujours accompagné tout au long de mon parcours universitaire. Que Dieu vous préserve et vous procure santé et longue vie.

À mes frères et sœurs,

Pour votre présence constante, vos encouragements et votre affection fraternelle.

À toute ma famille, mes proches et mes amis,

Qui ont toujours cru en moi et m'ont soutenu durant les moments les plus intenses de mes études.

Merci d'être toujours là pour moi.

Remerciements

Avant tout, je tiens à exprimer ma profonde gratitude à Dieu le Tout-Puissant pour m'avoir donné la force, la patience et la persévérance nécessaires pour mener à bien ce travail.

Je tiens à exprimer mes sincères remerciements à mon école d'accueil, l'Université de Nice Sophia Antipolis (UNICA) et l'École Marocaine des Sciences de l'Ingénieur (EMSI) pour m'avoir assuré une excellente formation académique et des bases solides pour entamer ma vie professionnelle.

J'adresse mes remerciements les plus chaleureux à mon encadrant pédagogique, **Mr. Marouan Skiti**, pour ses conseils méthodologiques avisés, sa disponibilité continue et la bienveillance dont il a fait preuve tout au long du suivi de ce projet de fin d'études.

Mes remerciements les plus vifs s'adressent à l'institution de mon stage, **CIH BANK**, pour m'avoir ouvert ses portes et permis de réaliser ce travail dans un environnement professionnel stimulant et formateur.

J'exprime toute ma reconnaissance à mes encadrants de stage chez CIH Bank, **Mr. Mohamed Agouzzal** (CDO - Chef du Data Office) et **Mr. Marouan Naoui** (Data Quality Analyst / Spécialiste Data Management & Gouvernance), pour leur accueil chaleureux, leur confiance, leur précieux encadrement et leur soutien technique rigoureux durant toute la durée de mon projet. Leurs directives éclairées m'ont grandement aidé à surmonter les difficultés techniques et à concevoir ce pipeline.

Enfin, je remercie l'ensemble des collaborateurs de CIH Bank et les membres de l'équipe Data Office pour leur esprit d'équipe, leur soutien et les échanges enrichissants que nous avons partagés. Que tous ceux qui ont contribué de près ou de loin à la réussite de ce projet trouvent ici l'expression de ma profonde gratitude.

Résumé

Dans un contexte de transformation digitale du secteur bancaire, la donnée constitue un levier stratégique essentiel pour la prise de décision, la gestion des risques et l'amélioration de la performance globale. Cependant, la qualité des données représente un enjeu majeur, car des données incomplètes, incohérentes ou erronées peuvent impacter significativement les processus métiers et les analyses.

Ce projet s'inscrit dans le cadre des initiatives de Data Governance au sein de CIH Bank. Il vise à concevoir et mettre en œuvre un pipeline automatisé de contrôle de la qualité des données, permettant d'assurer la fiabilité, la traçabilité et la cohérence des informations exploitées.

La solution proposée repose sur le traitement et le contrôle de données issues des systèmes opérationnels pour la base des tiers (qui alimentent le Data Lake) ainsi que de fichiers structurés, en intégrant des règles de contrôle couvrant plusieurs dimensions de la qualité des données telles que la complétude, la validité, l'unicité et l'intégrité.

L'automatisation des contrôles permet d'instaurer une surveillance continue de la qualité des données, avec la production d'indicateurs (KPI) facilitant le pilotage et la prise de décision. Ce projet contribue ainsi à renforcer la gouvernance des données et à soutenir les initiatives analytiques et décisionnelles de la banque.

Abstract

In the context of digital transformation within the banking sector, data has become a key strategic asset for decision-making, risk management, and overall performance improvement. However, data quality remains a major challenge, as incomplete, inconsistent, or erroneous data can significantly impact business processes and analytical outcomes.

This project is part of the Data Governance initiatives at CIH Bank. Its objective is to design and implement an automated data quality control pipeline to ensure the reliability, traceability, and consistency of the data being used.

The proposed solution is based on processing and controlling data from operational systems for the third-party database (which feed the Data Lake) as well as structured files, while applying a set of rules covering key data quality dimensions such as completeness, validity, uniqueness, and integrity.

By automating these controls, the solution enables continuous monitoring of data quality and the generation of key performance indicators (KPIs), supporting efficient data governance and informed decision-making. This project therefore contributes to strengthening the organization's data management framework and enhancing its analytical capabilities.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	ix
Introduction Générale	1
1 Présentation du cadre de projet	2
1.1 Présentation de l'organisme d'accueil	2
1.1.1 Missions de CIH BANK	2
1.1.2 Secteurs d'activité	3
1.1.3 Filiales	3
1.1.4 Actionnariat	3
1.1.5 Chiffres clés	3
1.1.6 Organigramme	4
1.1.7 Data Office	5
1.2 Étude de l'existant	6
1.2.1 Description de l'existant	6
1.2.2 Critique de l'existant	8
1.2.3 Solution proposée	8
1.3 Choix du modèle de développement	9
1.4 Planning prévisionnel	9
2 Analyse des Besoins et Spécification	11
2.1 Besoins fonctionnels	11
2.1.1 Automatisation et planification des contrôles	11
2.1.2 Configuration déclarative des règles de qualité	11
2.1.3 Historisation et traçabilité des indicateurs	12
2.1.4 Restitution visuelle et aide à la décision	12
2.2 Besoins non fonctionnels	12
2.2.1 Performance	12
2.2.2 Disponibilité	12
2.2.3 Maintenabilité	13
2.2.4 Sécurité	13
2.3 Modélisation des cas d'utilisation	13
2.3.1 Acteurs du système	13

2.3.2	Diagramme global des cas d'utilisation	14
2.3.3	Description textuelle des cas d'utilisation principaux	14
3	Conception de la Solution	17
3.1	Modélisation dynamique	17
3.1.1	Diagrammes de séquences	17
3.1.2	Diagrammes d'états-transitions	18
3.1.3	Diagrammes d'activités	18
3.2	Modélisation statique	19
3.2.1	Diagramme de classes	19
3.2.2	Modèle relationnel	19
3.2.3	Dictionnaire de données	20
3.2.4	Catalogue des règles de qualité	20
3.3	Architecture de l'application	23
3.3.1	Architecture logicielle (diagramme de composants)	23
3.3.2	Architecture matérielle (diagramme de déploiement)	25
3.4	Technologies de l'écosystème utilisées	26
4	Réalisation de la Solution	28
4.1	Environnement technique de développement	28
4.1.1	Environnement Hadoop et cluster	28
4.1.2	Installation de Soda Core	29
4.2	Implémentation du moteur DQ — Soda Core	29
4.2.1	Structure des fichiers du projet DQ	29
4.2.2	Fichier de configuration YAML — Règles KYC (<code>checks_kyc.yml</code>)	29
4.2.3	Fichier de configuration YAML — Règles CSP (<code>checks_csp.yml</code>)	30
4.2.4	Script Python principal (<code>dq_tiers_checks.py</code>)	31
4.3	Création des tables Hive de persistance	33
4.4	Intégration dans le DAG Airflow <code>dailyow22</code>	34
4.4.1	Ajout de la tâche DQ	34
4.4.2	Planification et monitoring	35
4.5	Création des vues Dremio	35
4.5.1	Vue <code>v.dq_scores_history</code>	35
4.5.2	Vue <code>v.dq_kpis_summary</code>	35
4.5.3	Vue <code>v.dq_anomalies_summary</code>	36
4.6	Réalisation du dashboard Tableau	36
4.6.1	Connexion Tableau à Dremio	36
4.6.2	Onglet 1 — Vue Exécutive	37
4.6.3	Onglet 2 — Détail KYC	37
4.6.4	Onglet 3 — Détail CSP	38
4.6.5	Onglet 4 — Anomalies et Remédiation	38

Table des figures

1	Chiffres clés de CIH BANK (31/12/2024)	4
2	Organigramme de CIH BANK	5
3	Architecture matérielle Big Data	7
4	Planning prévisionnel du projet	10
5	Diagramme global des cas d'utilisation de la solution DQ Tiers	14
6	Diagramme de séquences du pipeline de contrôle et restitution DQ	17
7	Diagramme d'états-transitions d'un profil tiers	18
8	Diagramme d'activités du workflow de remédiation DQ	18
9	Diagramme de classes unifié de la table de faits DQ	19
10	Diagramme de composants logiciels de la solution DQ	23
11	Diagramme de déploiement matériel de la solution DQ dans le cluster Big Data	25
12	Dashboard Tableau — Vue Exécutive : scores DQ globaux KYC et CSP . .	37
13	Dashboard Tableau — Détail KYC : score par colonne et dimension DQ .	37
14	Dashboard Tableau — Détail CSP : taux de nullité et valeurs parasites . .	38

Liste des tableaux

2	Informations générales sur le groupe CIH BANK [10]	2
3	Répartition du capital de CIH BANK [4]	3
4	Acteurs du système de Data Quality	14
5	Description du cas d'utilisation 01	15
6	Description du cas d'utilisation 02	15
7	Description du cas d'utilisation 03	16
8	Dictionnaire de la table de faits unique <code>dq_tiers_results_fact</code>	20
9	Catalogue des règles DQ — Données KYC (<code>ccm_npdescription</code>)	21
10	Catalogue des règles DQ — Données CSP (<code>ccm_job</code>)	22
11	Règles DQ — Fraîcheur et Pipeline	23
12	Versions des composants de l'environnement technique	28

Liste des abréviations

Abréviation	Signification
CIH	Crédit Immobilier et Hôtelier
CDO	Chief Data Officer
DWH	Data Warehouse
BI	Business Intelligence
ETL	Extract, Transform, Load
SQL	Structured Query Language
KPI	Key Performance Indicator
API	Application Programming Interface
CSV	Comma-Separated Values
ANRF	Autorité Nationale du Renseignement Financier
CNSS	Caisse Nationale de Sécurité Sociale
CSP	Catégories Socio-Professionnelles
KYC	Know Your Customer

Introduction Générale

Au cours des dernières années, le secteur bancaire a connu une transformation profonde sous l'impulsion de la digitalisation et de l'essor des technologies liées à la donnée. Dans ce contexte, les institutions financières s'appuient de plus en plus sur leurs systèmes d'information pour analyser les comportements clients, optimiser leurs processus et piloter leurs activités de manière efficace.

La donnée est ainsi devenue un actif stratégique. Toutefois, sa valeur dépend directement de sa qualité. Des données inexactes ou mal structurées peuvent engendrer des erreurs d'analyse, des décisions inadaptées et des risques importants, notamment dans des domaines sensibles tels que l'octroi de crédit ou la conformité réglementaire.

Face à ces enjeux, les organisations mettent en place des démarches de Data Governance visant à structurer, contrôler et valoriser leurs données. Parmi les axes majeurs de cette gouvernance figure la gestion de la qualité des données, qui repose sur la définition de règles de contrôle, leur automatisation et le suivi d'indicateurs de performance.

C'est dans ce cadre que s'inscrit le présent projet, réalisé au sein de CIH Bank. Il a pour objectif de concevoir et de mettre en œuvre une solution automatisée permettant de contrôler, mesurer et améliorer la qualité des données.

Le présent rapport est structuré comme suit : le premier chapitre présente l'organisme d'accueil, le deuxième expose le contexte et la problématique, le troisième décrit la conception de la solution, le quatrième détaille sa réalisation, et le dernier chapitre analyse les résultats obtenus.

Chapitre 1

Présentation du cadre de projet

Introduction du chapitre

Dans ce chapitre, nous présenterons une description générale de l'organisme d'accueil, CIH BANK, d'un point de vue organisationnel. Nous aborderons ensuite l'étude de l'existant, le choix du modèle de développement ainsi que le planning prévisionnel du projet, afin de mieux contextualiser notre travail de fin d'études.

1.1 Présentation de l'organisme d'accueil

Le Crédit Immobilier et Hôtelier (CIH), plus communément appelé CIH BANK, est une banque marocaine dont le siège social se trouve à Casablanca. Historiquement connu pour son activité dans le secteur de l'immobilier, le CIH propose désormais des services bancaires variés à ses clients, particuliers et professionnels. [10]

CIH BANK a été fondée en 1920 sous le nom de Caisse de Prêts Immobiliers du Maroc (CPIM). En 1967, le groupe a étendu son activité au secteur hôtelier et a changé de nom pour devenir le Crédit Immobilier et Hôtelier. [10]

Création	1920
Fondateur	L'État marocain
Introduction en bourse	1967
Forme juridique	Société anonyme
Slogan	La banque de demain dès aujourd'hui
Siège social	187, Avenue Hassan II, Casablanca, Maroc
Président Directeur Général (PDG)	Lotfi SEKKAT
Société mère	Caisse de Dépôt et de Gestion
Effectif	2 181
Site web	www.cihbank.ma

TABLE 2 – Informations générales sur le groupe CIH BANK [10]

1.1.1 Missions de CIH BANK

Spécialisé dans le financement immobilier et hôtelier, le CIH est une banque à vocation particulière :

- Elle est un partenaire des pouvoirs publics en matière de financement du logement depuis 1920 ,

- Elle finance les promoteurs immobiliers au Maroc, par deux canaux différents :
 - En amont : Par le financement des promoteurs publics et privés (acquisition et viabilisation des terrains ainsi que la construction des programmes immobiliers)
 - En aval : Par l’octroi de crédits au logement pour les particuliers acquéreurs, sans oublier les services et produits proposés par la banque pour sa clientèle.

1.1.2 Secteurs d’activité

Le groupe CIH BANK et ses filiales offrent une vaste gamme de produits et de services à ses clients, particuliers et professionnels :

- Services bancaires ,
- Prêts aux particuliers (prêts immobiliers, crédits à la consommation, etc.) ,
- Financement d’entreprises (prêts d’investissement et d’exploitation, crédit-bail, etc.) ,
- Assurance ,
- Salle des marchés.

1.1.3 Filiales

Le groupe CIH BANK est constitué de plusieurs filiales :

- SOFAC : entreprise spécialisée dans les solutions de crédit ,
- SOFASSUR : entreprise spécialisée dans les services d’assurance ,
- Maroc Leasing : entreprise de leasing qui propose des solutions de financement pour les véhicules, le BTP, l’immobilier et la santé [6] ,
- CIH Courtage : courtier en assurance ,
- Umnia Bank : banque participative ,
- AJAR INVEST : société de gestion des Organismes de Placement Collectif Immobilier (OPCI), dont CIH BANK détient 40 % des parts [1] .

1.1.4 Actionnariat

La répartition du capital du groupe CIH BANK est présentée dans le tableau 3 :

TABLE 3 – Répartition du capital de CIH BANK [4]

Actionnaire	Part
Massira Capital Management	55,66 %
Flottant en bourse	16,37 %
Atlanta Sanad	11,61 %
Caisse de Dépôt et de Gestion (CDG)	6,69 %
Régime Collectif d’Allocation de Retraite (RCAR)	5,21 %
Personnel de la banque	4,36 %
Groupe Holmarcom	0,12 %

1.1.5 Chiffres clés

La figure 1 présente les chiffres clés de CIH BANK.

Les chiffres clés de CIH BANK reflètent la situation financière et opérationnelle de l’institution. Avec un capital social de 3,149 milliards de dirhams et un total bilan de 141

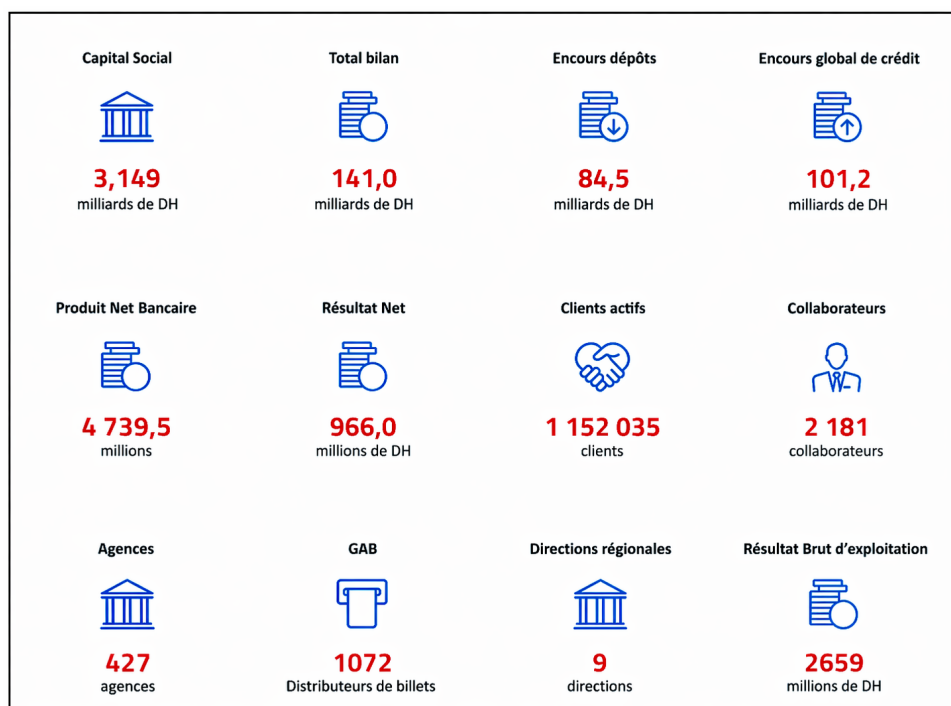


FIGURE 1 – Chiffres clés de CIH BANK (31/12/2024)

milliards de dirhams, la banque dispose d'une base financière significative. L'encours des dépôts s'élève à 84,5 milliards de dirhams, tandis que l'encours global de crédit atteint 101,2 milliards de dirhams. Le produit net bancaire est de 4 739,5 millions de dirhams, avec un résultat net de 966 millions de dirhams et un résultat brut d'exploitation de 2 659 millions de dirhams. La banque compte plus de 1,15 million de clients actifs, 2 181 collaborateurs, 427 agences, 1 072 guichets automatiques et 9 directions régionales. Ces indicateurs fournissent une vue d'ensemble de l'activité, de la structure et des résultats financiers de l'institution.

1.1.6 Organigramme

Structure hiérarchique de CIH BANK

La structure hiérarchique de CIH BANK s'articule autour d'une direction générale pilotée par le Président Directeur Général, appuyée par des fonctions de support stratégique et de contrôle (Audit, Conformité, Risques, Capital Humain, etc.) ainsi que diverses fonctions rattachées. L'activité commerciale et opérationnelle de la banque est quant à elle répartie en plusieurs pôles métiers majeurs : la Banque des Particuliers et Professionnels, la Banque de l'Entreprise et de l'Immobilier, la Banque de l'Investissement, le Financement & Recouvrement, et les Services à la Clientèle & Réseaux.

Parmi ces différentes directions, notre attention se porte plus particulièrement sur l'entité technologique qui encadre notre organisme d'accueil :

Fonctions technologiques et data

- Services Technologiques et Opérations : D. BENNOUNA ,
- Sécurité des Systèmes d'Information ,
- Qualité ,
- Pôle Systèmes d'Information : T. CHARKAOUI ,
- Data Management & Intelligence Artificielle :

L'organigramme global de CIH BANK est illustré dans la figure 2 ci-dessous.



Dans le cadre de notre stage, nous avons intégré le Data Office de CIH BANK, une entité dédiée à la gestion, au traitement et à la valorisation des données de la banque.

Le Data Office est aujourd’hui rattaché au pôle Services Technologiques et Opérations, plus précisément au sein de la direction Data Management et Intelligence Artificielle. Cette organisation reflète l’importance stratégique accordée à la donnée dans le pilotage des activités de la banque.

Cette entité regroupe plusieurs profils complémentaires intervenant sur l’ensemble du cycle de vie de la donnée, notamment :

- Les Data Engineers, en charge de l’ingestion, du traitement et de la mise à disposition des données,
- Les Data Analysts, responsables de l’analyse des données et de la production d’indicateurs décisionnels,
- Les Data Scientists, spécialisés dans la modélisation avancée et l’analyse prédictive,
- L’équipe Data Management, qui assure la gouvernance, la qualité et la structuration des données.

Les données sont collectées à partir des différents systèmes opérationnels de la banque, puis intégrées et centralisées au sein d’un Data Lake. Elles sont ensuite transformées et mises à disposition pour répondre aux besoins analytiques, réglementaires et métiers.

Dans ce cadre, le Data Office joue un rôle central dans la mise en place des pratiques de gestion de la qualité des données.

1.2 Étude de l’existant

1.2.1 Description de l’existant

Contexte et enjeux de la qualité des données bancaires

Dans l’écosystème financier contemporain, la transition vers une économie numérique a redéfini la nature des institutions bancaires. La donnée n’est plus un simple sous-produit de l’activité transactionnelle, mais s’impose comme une infrastructure essentielle sur laquelle reposent l’évaluation des risques, la conformité réglementaire (notamment BCBS 239 et le guide RDARR de la BCE) et les décisions stratégiques. Sur le plan économique, le coût de la non-qualité est très élevé. La « règle de dix » (*Rule of Ten*) stipule qu’il coûte dix fois plus cher de corriger une donnée en aval que de la valider directement à la source. Une gouvernance proactive apporte une valeur mesurable : optimisation des fonds propres, efficacité opérationnelle et sécurisation des décisions basées sur l’intelligence artificielle.

Le domaine Tiers : Cartographie et données

Historiquement, la vérification de la qualité des données au sein de CIH BANK était un processus réactif déclenché sur demande, ce qui a motivé la création de l’entité Data Management (DM). Notre projet s’inscrit dans cette volonté d’amélioration continue et cible plus particulièrement le **Référentiel Tiers**. Ce référentiel regroupe l’ensemble des informations relatives aux personnes physiques et morales en relation avec la banque.

Dans le cadre de l’ingestion des données vers le Data Lake, l’équipe travaille sur plus d’une dizaine de tables du domaine Tiers. À titre d’exemple et d’illustration, voici les cinq tables principales extraites du système opérant Oracle NOVA (CCM) qui structurent notre périmètre de base :

- **identity** : Données relatives aux pièces d’identité (type, numéro, date d’expiration),
- **legalentity** : Informations sur les personnes morales (raison sociale, forme juridique),

- **person** : Informations générales sur le statut du client (marché, agence, type),
- **professionaldescription** : Caractéristiques professionnelles et financières des entités,
- **np_description (Natural Person Description)** : Données d'état civil, d'identification et informations KYC (Know Your Customer) pour les personnes physiques.

Bien que notre étude de l'existant englobe ce vaste écosystème de tables, **l'implémentation technique, les algorithmes et les règles de Data Quality développés dans la suite de ce rapport se concentreront exclusivement sur la table np_description**. Cette table est éminemment stratégique car elle concentre les données KYC indispensables à l'identification réglementaire du client, telles que : le nom, le prénom, la date de naissance, la civilité, le genre, la nationalité, et l'identifiant FATCA (pour les clients américains).

Architecture et pipeline de données

L'ensemble des données ingérées au sein de CIH BANK, incluant celles du domaine Tiers, transite par une plateforme robuste centralisée s'appuyant sur une infrastructure Big Data de pointe. Tout l'écosystème analytique repose sur ce système unifié pour le traitement et la valorisation de la donnée.

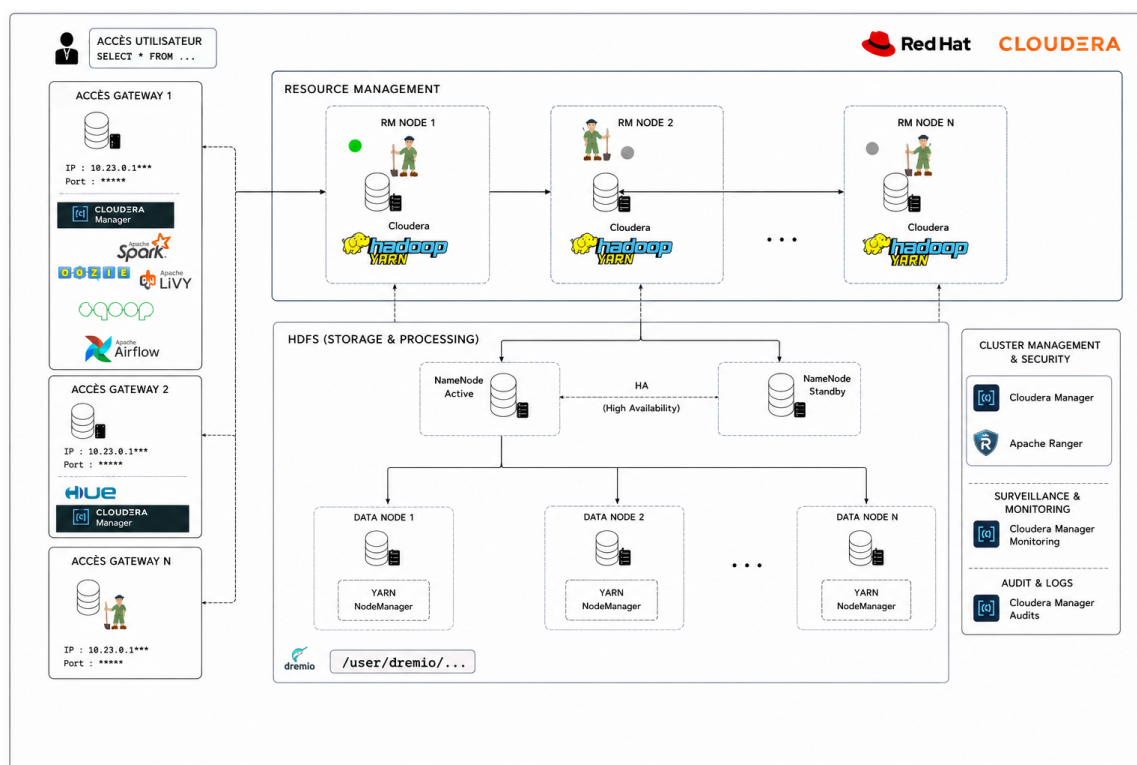


FIGURE 3 – Architecture matérielle Big Data

L'architecture matérielle et logicielle de cette plateforme de production s'articule autour d'un **cluster Cloudera et Dremio** composé de 8 nœuds (3 Masters, 3 Workers, 2 Gateways). Le flux global des données, de la source à l'exploitation, suit un cycle de vie standardisé en plusieurs étapes :

1. **Ingestion** : Les données brutes issues des systèmes opérants (tels qu'Oracle NOVA) sont ingérées massivement (souvent via Sqoop ou Kafka) et stockées dans la zone RAW du HDFS.

2. **Transformation** : Des jobs de traitement distribué (généralement via Apache Spark) nettoient, enrichissent et modélisent ces données brutes pour les charger dans la zone de STAGING.
3. **Virtualisation et Métadonnées** : Le moteur de données Dremio se connecte aux catalogues (Hive Metastore) pour virtualiser les tables. Il offre une couche sémantique puissante permettant de croiser et d'interroger les données sans les dupliquer.
4. **Exposition et Datamarts** : Les données transformées sont consolidées et exposées sous forme de Datamarts (ou vues métiers spécifiques). Ces ensembles de données prêts à l'emploi alimentent les outils de restitution (comme Tableau) pour répondre aux besoins analytiques des métiers.

1.2.2 Critique de l'existant

Bien que cette infrastructure de production Cloudera soit robuste, l'absence d'un contrôle automatisé de la qualité des données en amont pose plusieurs problèmes majeurs transverses à la plupart des tables du domaine Tiers. Pour illustrer notre démarche de remédiation, nous nous focaliserons sur la table `np_description`, qui contient des données particulièrement intéressantes concernant les personnes physiques (Natural Persons) :

- **Absence de validation à la source** : Les données transitent vers Dremio sans aucune validation intermédiaire. Des incohérences flagrantes persistent, comme des discordances entre le genre et la civilité du client, ou des dates de naissance remplies par défaut (ex : 01/01/XXXX) ,
- **Risques de non-conformité** : L'absence ou l'invalidité de certains champs critiques, tels que l'identifiant FATCA pour les clients concernés, expose la banque à des risques réglementaires et à d'éventuelles sanctions ,
- **Démarche réactive et manuelle** : Lorsqu'une anomalie est repérée par les métiers en bout de chaîne, la réconciliation implique asynchronement plusieurs équipes pour des corrections curatives chronophages, créant ainsi des processus inefficaces ,
- **Manque de traçabilité** : Aucune métrique n'est historisée. Les Data Stewards ne disposent d'aucun outil pour prioriser ou surveiller la santé des données KYC au quotidien.

1.2.3 Solution proposée

Pour pallier ces limites, nous proposons de concevoir et d'implémenter un **pipeline automatisé de contrôle de la qualité des données**, intégré directement dans la zone RAW de l'infrastructure Hadoop, avec un focus particulier sur la table `np_description`. Pour pallier ces limites, nous proposons de concevoir et d'implémenter un **pipeline automatisé de contrôle de la qualité des données**, intégré directement dans la zone RAW de l'infrastructure Hadoop.

La solution technique s'articulera autour des axes suivants :

- **Automatisation via Soda Core** : L'intégration de la bibliothèque Soda Core au sein des workflows Airflow existants permettra de valider systématiquement les données KYC (exhaustivité, unicité, validité de format, cohérence croisée) dès leur ingestion ,
- **Approche proactive** : En signalant les anomalies en zone RAW, nous empêchons l'exploitation de données erronées par les métiers, appliquant ainsi le principe de la *Rule of Ten* ,

- **Monitoring et Restitution** : Les résultats des contrôles et les anomalies seront persistés dans des tables Hive dédiées. Un tableau de bord interactif sera ensuite construit pour offrir aux Data Stewards une vue claire de la santé des tables du référentiel. Les données sensibles affichées dans les tableaux de bord seront systématiquement anonymisées ou masquées pour garantir la confidentialité et la sécurité des informations clients.

Cette solution permettra d'améliorer considérablement la gouvernance des données du domaine Tiers tout en optimisant le temps de traitement des anomalies.

1.3 Choix du modèle de développement

Dans le cadre de ce projet de fin d'études, plutôt que d'adopter un modèle purement séquentiel en cascade, nous avons opté pour une approche **itérative et incrémentale** s'inspirant des méthodes agiles.

Ce choix a été motivé par la complexité de l'écosystème Big Data existant et l'étendue du domaine Tiers. Il était impossible de livrer une solution parfaite en une seule fois. Le projet a donc été rythmé par la réalisation de *Proof of Concepts* (POC) et de *Minimum Viable Products* (MVP), permettant de valider les règles de qualité table par table, et étape par étape.

Cette démarche nous a permis de maintenir une collaboration continue avec les équipes métiers, et tout particulièrement avec le Data Steward du domaine Tiers et les équipes du Système d'Information (SI), via des points de synchronisation réguliers tout au long des six mois de stage.

1.4 Planning prévisionnel

Afin de mener à bien ce projet au sein du Data Office de CIH BANK, nous avons défini un planning de travail couvrant une période de six mois, du **9 février au 9 août 2026**.

Contrairement à un projet classique, l'avancement n'a pas été strictement linéaire. Les deux premiers mois ont été presque exclusivement dédiés à la compréhension fonctionnelle, technique, et à la documentation détaillée de l'architecture existante. Par la suite, les phases de conception, de réalisation des POCs, et de tests se sont superposées de manière itérative, accompagnées de réunions continues avec les métiers.

La figure 4 ci-dessous illustre ce déroulement sous forme d'un diagramme de Gantt, mettant en évidence la phase initiale de documentation, le cycle itératif de développement, et l'implication constante des acteurs métiers.

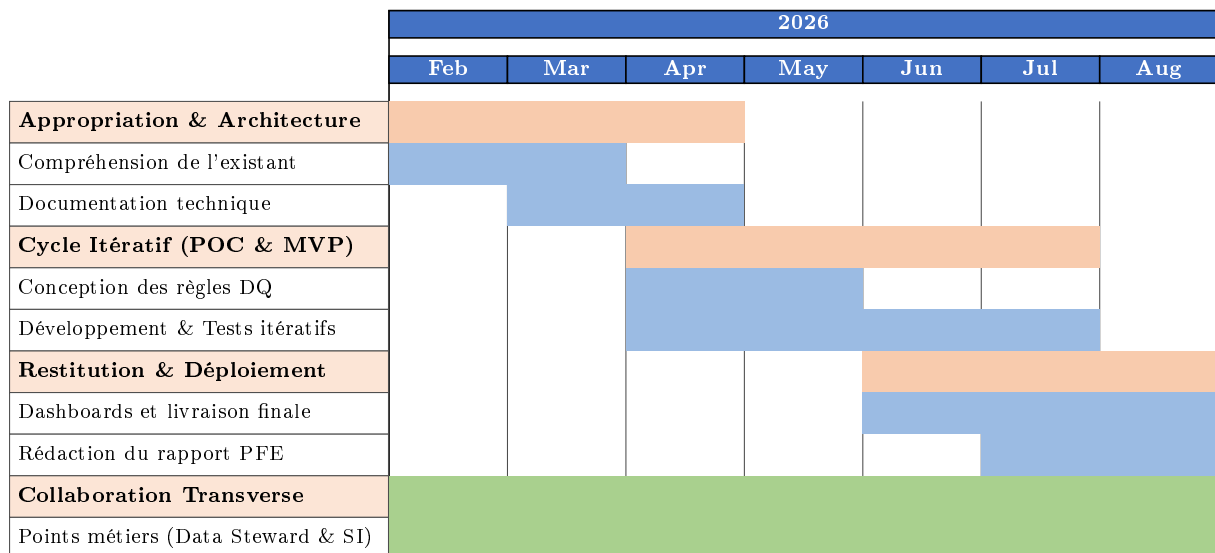


FIGURE 4 – Planning prévisionnel du projet

Conclusion du chapitre

Pour conclure, CIH BANK est une institution bancaire marocaine majeure, fortement structurée et engagée dans la valorisation de la donnée à travers son Data Office. L'étude de l'existant a permis de mettre en évidence les processus actuels de gestion des données et leurs limites, ce qui justifie la nécessité d'un pipeline automatisé de contrôle qualité. Ces éléments constituent la base de notre projet et nous permettent d'entamer sereinement la phase de spécification des besoins dans le chapitre suivant.

Chapitre 2

Analyse des Besoins et Spécification

Introduction du chapitre

Après avoir décrit le cadre général du projet et analysé le fonctionnement de la solution existante au sein de CIH BANK, ce chapitre se focalise sur la définition claire et rigoureuse des besoins. Nous y détaillons d’une part les besoins fonctionnels, c’est-à-dire les fonctionnalités attendues du pipeline de Data Quality, et d’autre part les besoins non fonctionnels, liés aux exigences techniques, de performance et de sécurité. Enfin, nous présentons la modélisation fonctionnelle à travers la spécification des cas d’utilisation et de leurs acteurs.

2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions que le système doit être capable de réaliser pour répondre aux problématiques de qualité des données identifiées sur le domaine Tiers. Ces problématiques, transverses à l’ensemble des tables du référentiel, seront illustrées par la table `np_description` et ses 40 colonnes (`name`, `firstname`, `civility`, `sex`, `birthdate`, `nationalitycountry`, `fatcaidentificationnumber`, etc.). Les besoins se répartissent en quatre fonctionnalités majeures :

2.1.1 Automatisation et planification des contrôles

- **Sous-besoin 1.1 : Déclenchement automatique** : Le système doit lancer l’évaluation de la qualité des données de manière planifiée, immédiatement après l’exécution complète du batch quotidien d’ingestion des tables sources Oracle NOVA (DAG Airflow `dailyow22` s’exécutant chaque soir à 22h00) ,
- **Sous-besoin 1.2 : Orchestration intégrée** : L’exécution du scan de qualité doit être orchestrée au sein d’Apache Airflow, permettant de suivre son statut d’exécution (Succès/Échec) et de consulter les logs en temps réel.

2.1.2 Configuration déclarative des règles de qualité

- **Sous-besoin 2.1 : Moteur de règles dynamique** : Le système doit permettre de définir les contrôles de qualité (Missing, Invalid, Freshness, Consistency, Uniqueness) dans des fichiers de règles déclaratifs séparés du code source ,
- **Sous-besoin 2.2 : Graduation des seuils et des alertes** : Pour chaque contrôle, le système doit pouvoir évaluer des seuils paramétrables (ex : taux de valeurs

manquantes inférieur à 5%) et générer un statut de warning ou d'error selon la gravité de l'écart ,

- **Sous-besoin 2.3 : Gestion des métadonnées** : Chaque règle doit être documentée avec des attributs métier (nom de la règle, dimension DQ associée, domaine KYC ou CSP, criticité, etc.).

2.1.3 Historisation et traçabilité des indicateurs

- **Sous-besoin 3.1 : Persistance journalière** : À chaque exécution, le système doit sauvegarder les métriques calculées et les résultats de chaque contrôle de qualité dans une table d'historique de scores (base Hive dédiée `dq_results`), indexée par date d'exécution ,
- **Sous-besoin 3.2 : Identification des anomalies (Remédiation)** : Le système doit capturer de manière détaillée la liste des enregistrements clients en anomalie en stockant leur identifiant technique (UUID) et leur radical (numéro client), ainsi que le libellé du contrôle en échec et sa criticité. Ces informations doivent être stockées dans une table d'anomalies pour actions correctives.

2.1.4 Restitution visuelle et aide à la décision

- **Sous-besoin 4.1 : Exposition des données** : Les résultats d'historisation de la qualité doivent être exposés de manière transparente sous forme de vues SQL pour les outils décisionnels ,
- **Sous-besoin 4.2 : Suivi de l'évolution temporelle** : Le tableau de bord doit permettre de suivre la tendance d'évolution des scores DQ dans le temps (30 derniers jours) ,
- **Sous-besoin 4.3 : Priorisation des remédiations** : Le tableau de bord doit fournir une liste opérationnelle des clients en anomalie triée par criticité décroissante, exportable par les Data Stewards pour traitement correctif dans le système source NOVA.

2.2 Besoins non fonctionnels

Les besoins non fonctionnels expriment les exigences qualitatives, opérationnelles et techniques que la solution doit respecter pour être viable et performante dans l'environnement de production de CIH BANK :

2.2.1 Performance

Le Référentiel Tiers de la banque regroupe **plusieurs millions** de dossiers clients (personnes physiques). Les règles de cohérence inter-colonnes ou de doublons nécessitent des jointures lourdes. Le moteur DQ doit être capable d'interroger et d'évaluer ce volume massif de données en un temps minimal (moins de 15 minutes) sans dégrader les ressources du Data Lake. Cela justifie l'utilisation d'un moteur de calcul distribué (Apache Spark) pour le traitement.

2.2.2 Disponibilité

L'exécution des contrôles de qualité ne doit pas bloquer la disponibilité des données pour les outils de reporting en cas d'anomalie. Si un seuil de qualité est franchi ou si le

scan échoue, le pipeline d'exposition (Dremio/Tableau) doit continuer à fonctionner en mode dégradé en signalant l'alerte. De plus, la tâche DQ doit s'exécuter même si certaines tables d'ingestion n'ont été chargées que partiellement.

2.2.3 Maintenabilité

L'ajout de nouvelles règles ou l'intégration de nouvelles colonnes du Référentiel Tiers (par exemple pour de nouveaux critères réglementaires ou pour les personnes morales) doit se faire sans modification du code Python principal de la solution. Les fichiers de configuration YAML et la structure des tables Hive de résultats doivent être conçus de manière générique pour supporter l'extensibilité.

2.2.4 Sécurité

Les détails d'anomalies de qualité des données contiennent des informations clients sensibles (`name`, `firstname`, `radicaux`). Seuls les collaborateurs habilités du Data Office (Data Stewards, Data Project) et les Data Owners des départements concernés doivent être autorisés à accéder aux détails des anomalies. La solution s'appuie sur les mécanismes de sécurité existants de la plateforme :

- **Authentification Kerberos** : L'ensemble des composants du cluster Cloudera (HDFS, Hive, Spark) sont sécurisés par Kerberos, garantissant l'identité de chaque service et utilisateur ,
- **Autorisation Apache Ranger** : Les politiques d'accès aux tables Hive et aux espaces HDFS sont gérées de manière centralisée via Ranger, permettant un contrôle fin par rôle ,
- **Annuaire LDAP / Active Directory** : L'authentification des utilisateurs Dremio et Tableau est déléguée à l'annuaire d'entreprise, assurant une gestion unifiée des identités.

2.3 Modélisation des cas d'utilisation

2.3.1 Acteurs du système

L'intégration de la solution de Data Quality au sein de CIH BANK repose sur une étroite collaboration entre les experts métier, l'équipe technique et les systèmes d'information. Le tableau 4 présente les cinq acteurs clés du système.

TABLE 4 – Acteurs du système de Data Quality

Acteur	Type	Rôle dans le système DQ
Équipe Data (DE / Data Steward)	Interne	Organise les ateliers d'alignement, formalise les règles dans les fichiers YAML Soda Core et implémente les flux techniques.
Métiers (Data Owners)	Humain	Experts fonctionnels qui définissent et valident les règles de qualité, consultent le dashboard et remédient aux anomalies.
Équipe SI	Appui	Valide la faisabilité technique des règles vis-à-vis du modèle physique Oracle NOVA.
Airflow (Orchestrateur)	Système	Déclenche automatiquement le scan DQ chaque soir après l'ingestion (DAG dailyow22).
Oracle NOVA (Système source)	Externe	Récepteur final des corrections de données opérées par les gestionnaires suite aux alertes DQ.

2.3.2 Diagramme global des cas d'utilisation

Le diagramme ci-dessous synthétise graphiquement les cas d'utilisation du système de Data Quality et les interactions avec les différents acteurs impliqués dans ce cycle de gouvernance.

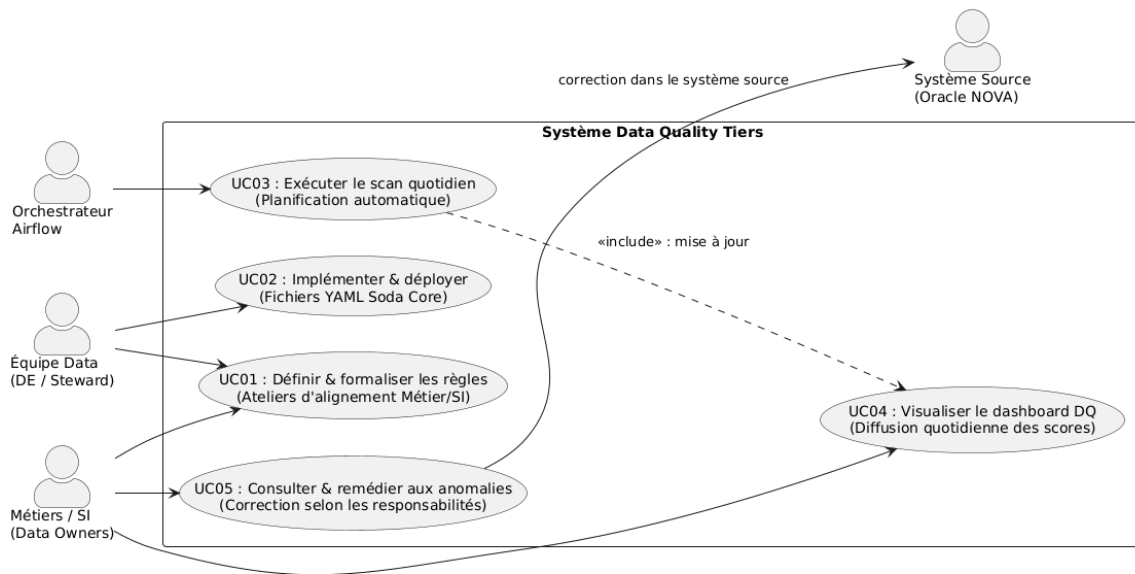


FIGURE 5 – Diagramme global des cas d'utilisation de la solution DQ Tiers

2.3.3 Description textuelle des cas d'utilisation principaux

Nous détaillons ci-dessous le déroulement des trois phases principales constituant le cycle de vie de la qualité des données de notre solution.

Cas d'utilisation 01 : Définir et formaliser les règles de qualité

TABLE 5 – Description du cas d'utilisation 01

Champ	Description
Titre	Définir et formaliser les règles de qualité
Acteurs	Équipe Data (Steward / DE), Métiers (Data Owners), Équipe SI
Préconditions	Cadrage fonctionnel du domaine Tiers défini
Scénario nominal	1. L'équipe Data organise des ateliers d'alignement avec les experts Métier et l'équipe SI. 2. Les parties s'accordent sur le catalogue des contrôles (complétude KYC, cohérence CSP) et les seuils de tolérance. 3. L'équipe Data consigne les spécifications dans un document de cadrage. 4. Le Data Engineer implémente les règles dans les fichiers YAML Soda Core.
Postconditions	Les règles de qualité sont formalisées et déployées dans le moteur Soda.

Cas d'utilisation 02 : Exécuter le scan de qualité (automatique)

TABLE 6 – Description du cas d'utilisation 02

Champ	Description
Titre	Exécuter le scan de qualité (automatique)
Acteur principal	Orchestrateur Airflow (Système)
Préconditions	Fin de l'ingestion quotidienne Sqoop des tables NOVA
Scénario nominal	1. Airflow lance automatiquement la tâche DQ de nuit. 2. Le moteur DQ Soda Core lit les fichiers YAML et exécute les requêtes sur Spark. 3. Les résultats agrégés et les détails des anomalies clients sont historisés dans la base Hive <code>dq_results</code>.
Postconditions	Les indicateurs et anomalies du jour sont prêts à être exposés.

Cas d'utilisation 03 : Consulter le dashboard et remédier aux anomalies

TABLE 7 – Description du cas d'utilisation 03

Champ	Description
Titre	Consulter le dashboard et remédier aux anomalies
Acteurs	Métiers (Parties prenantes), Data Stewards
Préconditions	Le scan DQ s'est exécuté de manière conforme
Scénario nominal	1. Le dashboard Tableau se rafraîchit automatiquement chaque matin. 2. Chaque partie prenante consulte le dashboard depuis son poste de travail. 3. Chaque acteur filtre et exporte la liste des anomalies de son périmètre. 4. Les acteurs procèdent à la correction des anomalies dans Oracle NOVA.
Postconditions	Les données corrigées dans NOVA sont ré-ingérées lors du prochain batch et le score DQ s'améliore automatiquement.

Conclusion du chapitre

Ce chapitre a permis de formaliser les besoins fonctionnels et non fonctionnels, définissant de manière précise les objectifs techniques et opérationnels du pipeline de qualité des données. La modélisation des acteurs et des cas d'utilisation reflète fidèlement le cycle de gouvernance de CIH BANK, où chaque partie prenante collabore d'abord à la définition des contrôles, puis intervient individuellement sur son périmètre pour corriger les anomalies identifiées au quotidien. Fort de ces spécifications, le chapitre suivant présente la conception globale et détaillée de la solution.

Chapitre 3

Conception de la Solution

Introduction du chapitre

Après avoir défini les besoins fonctionnels et non fonctionnels de notre système de contrôle de la qualité des données, ce chapitre présente la conception détaillée de la solution. Conformément aux exigences du Template 2, nous décrivons dans un premier temps la modélisation dynamique à travers des diagrammes de séquences, d'états-transitions et d'activités. Dans un second temps, nous présentons la modélisation statique comprenant le diagramme de classes, le modèle relationnel, le dictionnaire de données et le catalogue complet des règles de qualité. Enfin, nous détaillons l'architecture logicielle (diagramme de composants) et matérielle (diagramme de déploiement) de l'application dans son environnement CIH BANK.

3.1 Modélisation dynamique

La modélisation dynamique permet de représenter les interactions temporelles entre les différents composants du système et l'évolution des états des données au cours de l'exécution du processus de contrôle.

3.1.1 Diagrammes de séquences

Le diagramme de séquences de la figure 6 détaille le flux d'exécution séquentiel d'un scan de qualité de données. Il illustre comment la tâche DQ intégrée dans Airflow coordonne le traitement entre la zone HDFS RAW, le moteur Soda Core et la base Hive de résultats, jusqu'à l'exposition finale vers Tableau.

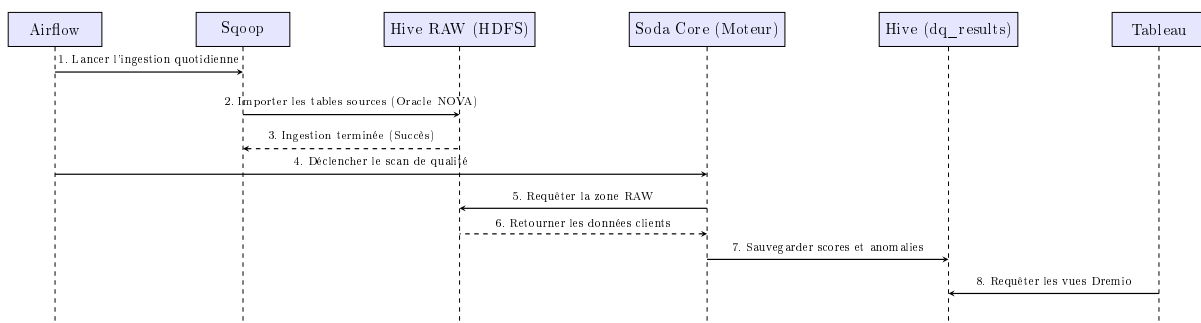


FIGURE 6 – Diagramme de séquences du pipeline de contrôle et restitution DQ

3.1.2 Diagrammes d'états-transitions

Le diagramme d'états-transitions de la figure 7 modélise les différents états par lesquels passe le dossier d'un tiers physique dans le cycle d'évaluation de la qualité des données, notamment lors de la détection et du traitement d'une anomalie.

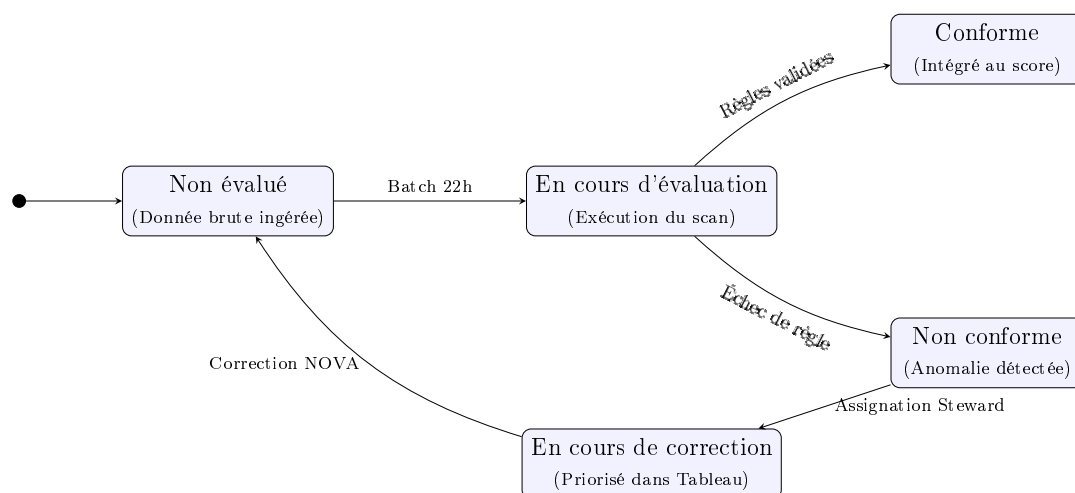


FIGURE 7 – Diagramme d'états-transitions d'un profil tiers

3.1.3 Diagrammes d'activités

Le diagramme d'activités présenté dans la figure 8 décrit le processus de bout en bout de traitement de la qualité, depuis le chargement initial des données en zone RAW HDFS jusqu'à la remédiation effective dans l'application source Oracle NOVA.

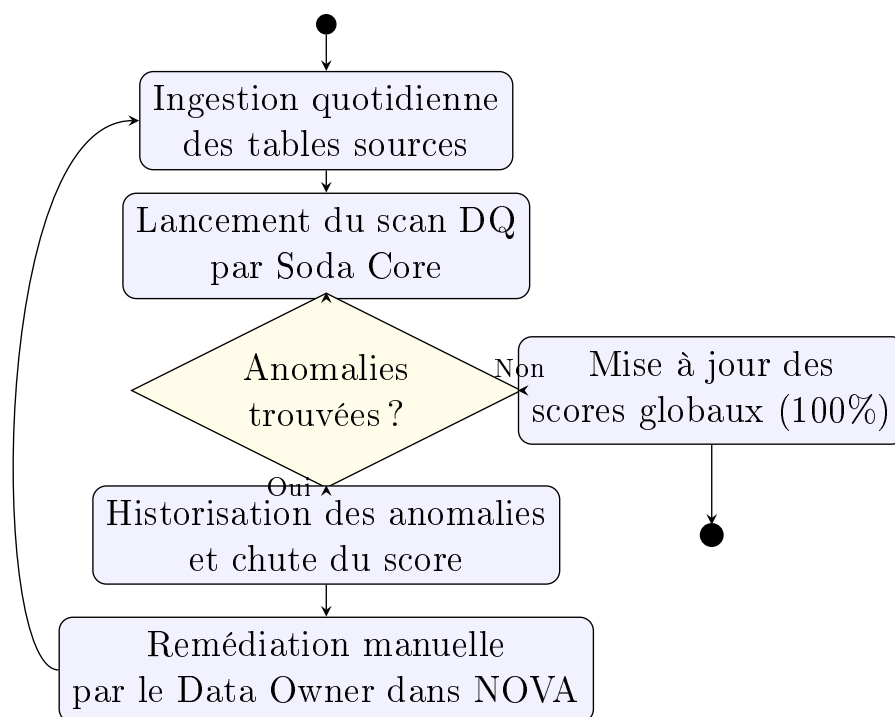


FIGURE 8 – Diagramme d'activités du workflow de remédiation DQ

3.2 Modélisation statique

La modélisation statique définit la structure fixe du système, ses composants de données, le schéma relationnel de stockage de l'historique de qualité, ainsi que le dictionnaire des métadonnées associées.

3.2.1 Diagramme de classes

Le diagramme de classes de la figure 9 représente l'entité logique unique qui encapsule l'ensemble des résultats d'évaluation de la qualité des données (table de faits dénormalisée). Cette modélisation unifiée simplifie l'architecture logique en regroupant les informations sur les règles, les volumes de lignes vérifiées et non conformes, les scores et le temps d'exécution.

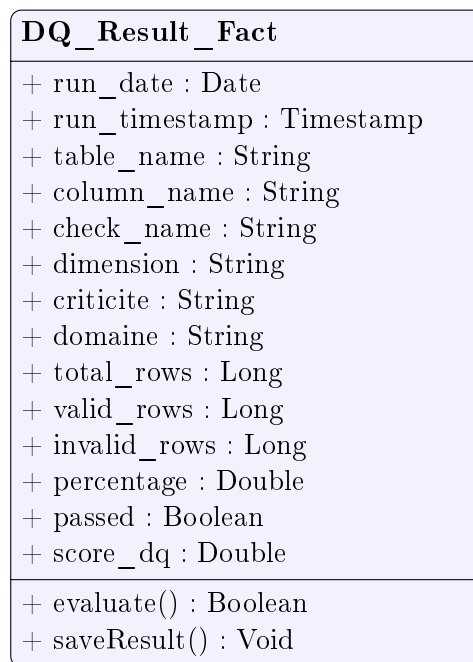


FIGURE 9 – Diagramme de classes unifié de la table de faits DQ

3.2.2 Modèle relationnel

Pour persister les résultats d'évaluation de la qualité des données de manière quotidienne, nous concevons un schéma physique relationnel indépendant au sein de la base de données Hive. Ce schéma convertit la classe de modélisation logique en une unique table de faits dénormalisée :

- **dq_tiers_results_fact** : Table unique d'historisation consolidée des indicateurs et scores de qualité par exécution, par table et par règle.

Le passage du modèle logique au modèle physique relationnel s'opère selon les règles classiques de traduction :

- La classe unique se traduit par une table de faits relationnelle dénormalisée ,
- Les attributs se traduisent par des colonnes typées avec gestion de la nullité ,
- Le partitionnement est appliqué sur la colonne de date (**run_date**) pour optimiser les performances de requêtage temporel et simplifier le chargement quotidien.

Le schéma relationnel textuel s'établit comme suit :

- **dq_tiers_results_fact** (`[run_date, table_name, column_name, check_name]`, `run_timestamp`, `dag_name`, `dimension`, `criticite`, `domaine`, `total_rows`, `valid_rows`, `invalid_rows`, `percentage`, `passed`, `score_dq`).

3.2.3 Dictionnaire de données

Nous détaillons ci-dessous la structure physique et le dictionnaire de données de la table de faits consolidée de persistance.

TABLE 8 – Dictionnaire de la table de faits unique **dq_tiers_results_fact**

Colonne	Type	Clé	Description / Rôle
<code>run_date</code>	DATE	PK (Part.)	Date d'exécution du contrôle de qualité.
<code>run_timestamp</code>	TIMESTAMP	-	Horodatage précis de l'exécution (autour de minuit).
<code>table_name</code>	STRING	PK	Table Hive contrôlée en zone RAW.
<code>column_name</code>	STRING	PK	Nom de la colonne soumise à la règle de qualité.
<code>check_name</code>	STRING	PK	Nom descriptif unique de la règle (ex. : <code>check_genre_civilite</code>).
<code>dag_name</code>	STRING	-	Nom du DAG Airflow d'orchestration (<code>dailyow22</code>).
<code>dimension</code>	STRING	-	Dimension DQ (<code>completeness</code> , <code>validity</code> , <code>consistency</code> , <code>uniqueness</code> , <code>freshness</code>).
<code>criticite</code>	STRING	-	Niveau de criticité du contrôle (<code>CRITIQUE</code> , <code>HAUTE</code> , <code>MOYENNE</code>).
<code>domaine</code>	STRING	-	Domaine métier visé (<code>KYC</code> ou <code>CSP</code>).
<code>total_rows</code>	BIGINT	-	Nombre total d'enregistrements clients vérifiés lors du scan.
<code>valid_rows</code>	BIGINT	-	Nombre d'enregistrements conformes validant la règle.
<code>invalid_rows</code>	BIGINT	-	Nombre d'enregistrements non conformes (anomalies).
<code>percentage</code>	DOUBLE	-	Taux de conformité en pourcentage (calculé par : <code>valid_rows / total_rows * 100</code>).
<code>passed</code>	BOOLEAN	-	Indique si le test a été validé (<code>TRUE</code> ou <code>FALSE</code>).
<code>score_dq</code>	DOUBLE	-	Score de qualité normalisé (entre 0.0 et 100.0).

3.2.4 Catalogue des règles de qualité

Les contrôles de qualité s'appliquent sur deux catégories logiques de données du domaine Tiers : les informations KYC (Know Your Customer) et les données CSP (Socio-Professionnelles).

L'établissement de ce catalogue de règles de qualité découle d'une analyse exploratoire approfondie menée sur un échantillon réel de 5 000 dossiers de personnes physiques issus de la table `ccm_npdescription`. Cette analyse a révélé des anomalies typiques et récurrentes

de saisie au sein du système opérant Oracle NOVA, ce qui a directement guidé le choix et le paramétrage de nos dimensions de contrôle :

- **Le phénomène des dates de naissance génériques** : Une forte concentration de dates de naissance au premier janvier (01/01/YYYY) a été identifiée (par exemple 1947-01-01, 1961-01-01), traduisant une saisie par défaut dans NOVA lorsque seule l'année de naissance est connue. Cela justifie notre règle de validité sur `birthdate` avec un seuil de tolérance de 15% ,
- **Le codage numérique des villes** : Le champ `birthcity` contient des codes numériques (comme 787) au lieu de libellés textuels, nécessitant un contrôle de validité par rapport au référentiel des villes pour assurer la cohérence et la résolubilité des codes ,
- **La cohérence conditionnelle FATCA** : L'identifiant FATCA (`fatcaidentificationnumber`) est majoritairement nul, ce qui est cohérent pour les citoyens marocains (`nationalitycountry` = MA), mais doit être obligatoirement renseigné pour les clients ayant la nationalité ou une double nationalité américaine.

Le catalogue complet est défini ci-dessous. Pour les données CSP, l'intégralité des colonnes de la table `ccm_job` fait obligatoirement l'objet d'un contrôle de **Complétude**. Des dimensions complémentaires de **Validité**, de **Cohérence** et d'**Unicité** sont ajoutées pour maximiser la robustesse de l'analyse opérationnelle.

TABLE 9 – Catalogue des règles DQ — Données KYC (`ccm_npdescription`)

Colonne(s)	Dimension	Description Règle / Seuil	Criticité	Action corrective
<code>name</code>	Complétude	0 valeur nulle tolérée	CRITIQUE	Audit dossiers - Blocage saisie agence
<code>firstname</code>	Complétude	0 valeur nulle tolérée	CRITIQUE	Audit dossiers - Blocage saisie agence
<code>name</code> , <code>firstname</code>	Validité	Longueur ≥ 2 , aucun chiffre ou caractère spécial	HAUTE	Nettoyage et normalisation dans NOVA
<code>birthdate</code>	Complétude	0 valeur nulle tolérée	CRITIQUE	Saisie obligatoire avec justificatif officiel
<code>birthdate</code>	Validité	Taux de dates génériques au 01/01 < 15%	HAUTE	Recherche de la date réelle sur pièce d'identité
<code>civility</code>	Validité	Valeurs restreintes à (M., MME, MLLE)	HAUTE	Restauration de la valeur selon pièces fournies
<code>sex</code>	Validité	Valeurs restreintes à (M, F)	HAUTE	Restauration de la valeur selon pièces fournies
<code>sex</code> + <code>civility</code>	Cohérence	Genre M $\neq$ MME/MLLE ; Genre F $\neq$ M.	CRITIQUE	Résolution manuelle des incohérences de saisie
<code>nationality_country</code>	Complétude	0 valeur nulle tolérée	HAUTE	Renseigner le code pays de nationalité
<code>fatca_identification_number</code>	Cohérence	Si nationalité ou double nationalité = US, 0 valeur nulle	CRITIQUE	Déclaration réglementaire FATCA immédiate

Colonne(s)	Dimension	Description Règle / Seuil	Criticité	Action corrective
birth_city	Validité	Doit exister dans le référentiel des villes BAM	MOYENNE	Mapping et correction via la table de transcodage

TABLE 10 – Catalogue des règles DQ — Données CSP (ccm_job)

Colonne(s)	Dimension	Description Règle / Seuil	Criticité	Action corrective
professional_sector	Complétude	Taux de valeurs nulles < 5%	HAUTE	Renseigner le secteur d'activité de l'employeur
professional_sector	Validité	Code numérique valide dans le référentiel sectoriel	MOYENNE	Transcodage vers les codes officiels BAM
occupation	Complétude	Taux de valeurs nulles < 5%	HAUTE	Préciser la profession exacte du client
socioprofessional_category	Complétude	Taux de valeurs nulles < 5%	HAUTE	Renseigner la catégorie socioprofessionnelle (CSP)
professional_situation	Complétude	Taux de valeurs nulles < 5%	HAUTE	Renseigner le statut de la situation professionnelle
professional_situation	Validité	Valeurs restreintes à (SAL, FON, TRI, AUT, EMP, NDE)	HAUTE	Remplacer par un code référentiel valide
employer	Complétude	Taux de valeurs nulles < 10%	HAUTE	Renseigner le nom de l'entreprise employeuse
employer	Validité	Détection des valeurs parasites ('AUTRE', 'LUI MEME')	HAUTE	Normaliser : mapper 'LUI MEME' vers auto-entrepreneur
employer + professional_situation	Cohérence	Si situation = NDE (sans emploi) ou TRI (retraité), employeur doit être NULL	HAUTE	Annuler le nom d'employeur incohérent dans NOVA
politically_exposed	Complétude	0 valeur nulle tolérée (PEP obligatoire)	CRITIQUE	Renseigner la déclaration d'exposition politique
ordnum	Unicité	Un seul enregistrement actif principal (main=1) par tiers	HAUTE	Éliminer les contrats de travail multiples actifs

Colonne(s)	Dimension	Description Règle / Seuil	Criticité	Action corrective
batch_ datetime	Fraîcheur	Retard d'ingestion HDFS RAW < 26h	HAUTE	Alerte infrastructure Big Data et relance Sqoop
code_segment	Cohérence	Cohérence entre segment client et code_marche	HAUTE	Recalculer le segment par défaut

TABLE 11 – Règles DQ — Fraîcheur et Pipeline

3.3 Architecture de l'application

L'architecture présente la structure logique de la solution DQ et son déploiement physique sur l'infrastructure de production de CIH BANK.

3.3.1 Architecture logicielle (diagramme de composants)

L'architecture logicielle de notre solution s'articule autour de modules découplés. Le diagramme de la figure 10 illustre les interactions entre les modules d'ingestion, de stockage, de contrôle, de virtualisation et de restitution de la solution.

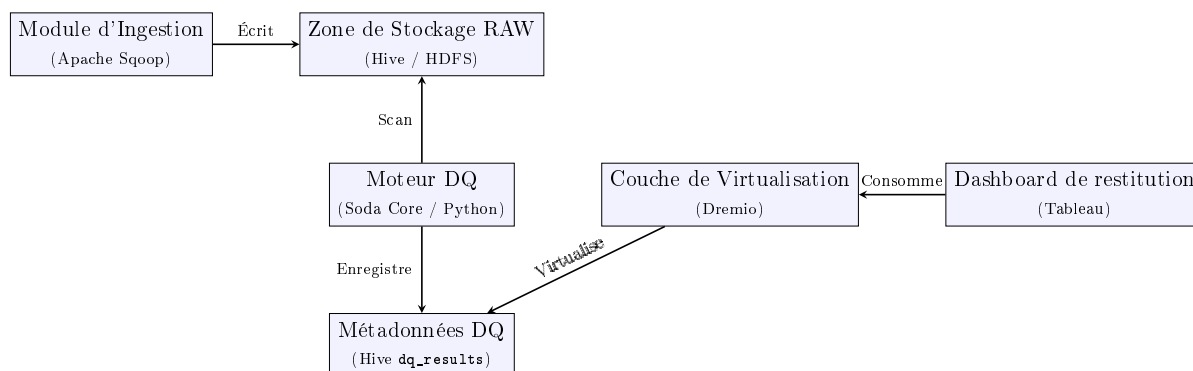


FIGURE 10 – Diagramme de composants logiciels de la solution DQ

Les composants clés de cette architecture logicielle sont détaillés ci-dessous :

Conception du moteur DQ — Soda Core

Le moteur de contrôle s'appuie sur le framework open source Soda Core, spécifiquement la bibliothèque Python `soda-core` et son connecteur Spark `soda-core-spark`. Afin d'assurer une exécution optimale dans notre écosystème Big Data, la conception logicielle repose sur les principes d'intégration suivants :

- **Encapsulation PySpark et Vues Temporaires** : Le script d'exécution principal (`dq_tiers_checks.py`) initialise une session Spark locale. Pour chaque table de la zone RAW à contrôler (ex : `ccm_npdescription` et `ccm_job`), la session Spark charge la table sous forme de DataFrame PySpark, puis l'enregistre en tant que vue temporaire en mémoire via la méthode `createOrReplaceTempView()`. Cela permet à Soda Core d'exécuter l'analyse de qualité directement sur ces vues mémoire sans poser de verrous ou d'accès concurrents sur les tables Hive physiques ,

- **Définition déclarative des règles (SodaCL)** : Les contrôles de qualité sont externalisés dans des fichiers de règles YAML exploitant le langage déclaratif SodaCL. Les fichiers sont séparés logiquement par domaine : `checks_kyc.yml` pour le KYC, `checks_csp.yml` pour les données socio-professionnelles, et `checks_pipeline.yml` pour les indicateurs techniques et la fraîcheur ,
- **Extraction programmatique et typage des métadonnées** : Après l'exécution du scan, le script Python exploite les structures d'API de Soda Core (l'objet `Scan`) pour parcourir la liste des vérifications évaluées. Pour chaque règle, le script extrait dynamiquement le résultat (succès/échec), la valeur de la métrique, le seuil, la dimension (Completeness, Validity, Consistency), le domaine et la criticité (définis en tant qu'attributs personnalisés dans le YAML). Un score de conformité sur 100 est attribué pour chaque règle ,
- **Persistance partitionnée des anomalies** : Pour toute règle non respectée ayant le statut `fail`, le script interroge la propriété `failed_rows_query` générée dynamiquement par Soda Core. Cette requête SQL est exécutée au moyen de Spark SQL pour isoler les identifiants uniques (`oid`) et les radicaux clients associés aux anomalies. Ces dossiers non conformes sont structurés dans un DataFrame puis ajoutés en mode append dans la table partitionnée Hive `dq_results.dq_tiers_anomalies`, tandis que le résumé quotidien des scores alimente la table `dq_results.dq_tiers_scores`.

Conception de l'orchestration Airflow

La planification, le déclenchement et le monitoring des scans de qualité sont entièrement automatisés par Apache Airflow au moyen d'un unique DAG quotidien nommé `dailyow22`. La tâche de Data Quality est conçue comme une étape post-ingestion. Elle s'exécute immédiatement après le succès des tâches d'ingestion Sqoop qui rapatrient les tables sources NOVA vers le Data Lake. Afin de garantir la surveillance du pipeline technique, la tâche de qualité est configurée avec la règle de déclenchement `TriggerRule.ALL_DONE`. Cela assure que même si une tâche d'ingestion non critique échoue, les contrôles de qualité s'exécutent sur les tables correctement importées, ce qui évite d'aveugler le monitoring en cas de panne technique partielle.

Conception des vues Dremio DQ

Pour exposer les indicateurs de qualité stockés dans Hive vers l'outil BI Tableau sans déplacement physique de données, nous concevons une couche de virtualisation via Dremio dans un schéma dédié nommé `dq-tiers`. Dremio se connecte directement au catalogue Hive Metastore et expose trois vues SQL virtuelles :

- `v.dq_scores_history` : Expose l'historique temporel des scores calculés par règle et par domaine pour alimenter les courbes de tendance temporelle ,
- `v.dq_kpis_summary` : Fournit une agrégation quotidienne des taux de conformité par dimension DQ ,
- `v.dq_anomalies_detail` : Liste les radicaux des clients en anomalie avec le libellé exact de l'erreur pour la remédiation.

Cette virtualisation garantit un temps de réponse inférieur à la seconde lors des interactions de l'utilisateur sur Tableau, grâce aux accélérations matérielles (réflexions de données) de Dremio.

Conception du dashboard Tableau

La restitution visuelle est structurée dans un dashboard Tableau interactif composé de quatre onglets fonctionnels :

1. **Vue Exécutive** : Présente les KPIs de conformité globaux et les alertes critiques pour les Data Owners et la Direction ,
2. **Détail KYC** : Permet une analyse fine par colonne et par dimension de qualité sur le domaine KYC ,
3. **Détail CSP** : Restitue les scores de conformité socioprofessionnelle et l'évaluation des valeurs parasites ,
4. **Anomalies & Remédiation** : Liste paginée contenant le radical et le détail de l'anomalie, exportable en Excel pour le traitement correctif par les Data Stewards.

Pour des raisons évidentes de sécurité et de conformité réglementaire (notamment les directives de Bank Al-Maghrib et la protection des données personnelles), toutes les données nominatives et les radicaux clients affichés dans les onglets opérationnels font l'objet d'un masquage ou d'un floutage systématique, garantissant la stricte confidentialité des informations sensibles.

3.3.2 Architecture matérielle (diagramme de déploiement)

Le diagramme de déploiement de la figure 11 modélise l'architecture système et l'infrastructure distribuée de production sur laquelle est déployée notre solution au sein de CIH BANK.

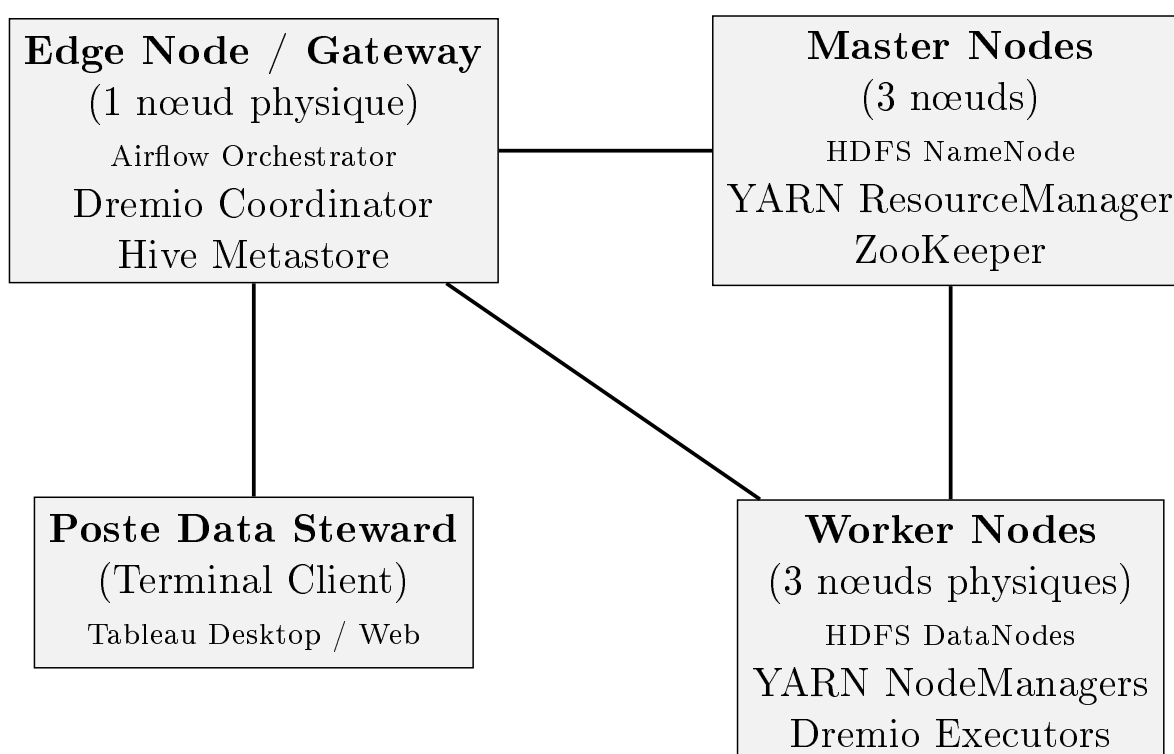


FIGURE 11 – Diagramme de déploiement matériel de la solution DQ dans le cluster Big Data

Le cluster Big Data Cloudera CDP 7.1.7 de production de CIH BANK est structuré autour de 8 nœuds distribués, configurés de la manière suivante :

- **1 Nœud Edge (Gateway) principal** : Ce serveur sert de point d'entrée pour l'administration et l'orchestration des jobs. Il héberge les workers d'Apache Airflow, le coordinateur Dremio, ainsi que le service Hive Metastore. C'est sur ce nœud qu'est installé l'environnement virtuel Python contenant les packages `soda-core` et `soda-core-spark` ,
- **3 Nœuds Masters** : Ils assurent la haute disponibilité du stockage (HDFS NameNodes configurés en Actif/Standby et JournalNodes) et la coordination de l'ordonnancement distribué. Ils hébergent également le YARN ResourceManager pour la répartition des ressources du cluster ,
- **3 Nœuds Workers** : Ces serveurs gèrent le stockage local physique sous HDFS (DataNodes) et disposent des YARN NodeManagers pour allouer des conteneurs de calcul lors de l'exécution distribuée de nos requêtes PySpark et Dremio Executors.

Orchestration technique et protocole de sécurité Kerberos

Afin de sécuriser les accès aux données confidentielles du domaine Tiers de CIH BANK, l'exécution de la solution de qualité s'intègre au protocole de sécurité global du cluster :

- **Authentification Kerberos** : Le cluster Cloudera étant sécurisé par Kerberos, l'autorisation d'accès aux répertoires HDFS RAW nécessite un ticket d'authentification valide. Avant de lancer le traitement, la tâche Airflow utilise le fichier keytab chiffré (`tiers_dq.keytab`) associé au compte de service du domaine Tiers (`tiers_dq_service`) pour générer et renouveler automatiquement le ticket Kerberos via la commande système `kinit` ,
- **Soumission du job Spark sur YARN** : Le script Python principal est exécuté en mode `spark-submit` avec le gestionnaire de ressources YARN configuré en mode `client` depuis le nœud Edge. Le driver Spark s'exécute ainsi localement sur le Gateway Node (dans le process Airflow), tandis que YARN ResourceManager alloue de manière distribuée des exécuteurs Spark au sein des nœuds Workers. Ces exécuteurs lisent et traitent en parallèle les partitions des fichiers HDFS de la zone RAW Hive.

3.4 Technologies de l'écosystème utilisées

Pour s'insérer au mieux au sein du système d'information décisionnel et Big Data de CIH BANK, notre solution exploite les briques technologiques de l'écosystème du Data Lake. Ces technologies sont résumées ci-dessous.



Oracle NOVA — Système opérant source

Base de données relationnelle hébergeant le Core Banking de la banque. Il s'agit du point d'entrée unique des données des clients physiques et moraux (Référentiel Tiers) et de la cible finale de correction.



Apache Sqoop — Ingestion batch Oracle vers HDFS

Technologie d'ingestion permettant le transfert en masse des tables NOVA vers l'infrastructure Hadoop de manière planifiée.



Apache Airflow — Orchestrateur de tâches

Outil d'orchestration permettant de planifier, ordonnancer et superviser l'exécution du flux d'ingestion et des scans de qualité de manière centralisée.



HDFS / Apache Hive — Plateforme de stockage et de requêtage
HDFS assure le stockage persistant distribué tandis que Hive structure ces répertoires sous forme de tables SQL relationnelles cibles pour nos contrôles.



Soda Core — Moteur de contrôle de la qualité (Python)
Framework de validation de données déclaratif basé sur des fichiers YAML de règles de gestion et exécuté programmatiquement sur les tables Hive RAW.



Dremio — Moteur de virtualisation et d'exposition
Permet de virtualiser les tables de résultats Hive et d'exposer des vues SQL rapides et sécurisées, éliminant le besoin de dupliquer ou de transférer physiquement les volumes de données.



Tableau — Outil d'analyse décisionnelle et Visualisation
Solution BI standard de CIH BANK, connectée en direct à Dremio pour fournir un reporting réactif sur la qualité des tiers à l'intention des Data Owners.

Conclusion du chapitre

Ce chapitre a décrit la conception complète de la solution de contrôle de la qualité du domaine Tiers. Nous avons structuré la dynamique d'exécution de nos scans et le cycle de vie des anomalies identifiées. La modélisation statique a fourni le diagramme de classes de nos indicateurs, le schéma relationnel et le catalogue complet des règles KYC et CSP de la solution. Enfin, l'architecture logicielle (composants) et matérielle (déploiement) a été détaillée au sein de l'environnement de production. Le chapitre suivant présente l'implémentation physique et les résultats opérationnels obtenus.

Chapitre 4

Réalisation de la Solution

Introduction

Ce chapitre présente la réalisation concrète du pipeline de Data Quality sur le Référentiel Tiers de CIH Bank. Tous les contrôles Soda Core s'effectuent exclusivement sur la **zone RAW** du Data Lake (base Hive `raw_nova_referentieltiers`), qui constitue la copie exacte et non transformée des tables `CCM_*` du système opérant Oracle NOVA. Nous détaillerons successivement l'environnement technique de développement, l'implémentation du script Soda Core, la création des tables Hive de persistance, l'intégration dans le DAG Airflow `dailyow22`, la mise en place des vues Dremio et la réalisation du dashboard Tableau.

4.1 Environnement technique de développement

4.1.1 Environnement Hadoop et cluster

Le développement et les tests ont été réalisés dans l'environnement Hadoop de CIH Bank. L'accès aux tables Hive et aux fichiers HDFS est effectué via les interfaces disponibles dans l'environnement Data Office, en utilisant les comptes de service existants du domaine Tiers.

Les versions des composants utilisés dans l'environnement de production sont les suivantes :

Composant	Version
Apache Hadoop (HDFS)	Cluster CIH Bank
Apache Hive	Cluster CIH Bank
Apache Spark	Cluster CIH Bank
Apache Airflow	Cluster CIH Bank
Dremio	Instance CIH Bank
Python	3.x
Soda Core	soda-core-spark
Tableau	Tableau Desktop / Server CIH Bank

TABLE 12 – Versions des composants de l'environnement technique

4.1.2 Installation de Soda Core

Soda Core est installé dans l'environnement Python accessible depuis les workers Airflow. Le connecteur retenu est `soda-core-spark`, qui permet à Soda Core d'utiliser une `SparkSession` existante comme moteur d'exécution pour interroger les tables Hive de la zone RAW.

```
1 pip install soda-core-spark
```

4.2 Implémentation du moteur DQ — Soda Core

4.2.1 Structure des fichiers du projet DQ

Le projet DQ est organisé selon l'arborescence suivante :

```
1 dq_tiers/
2 |-- dq_tiers_checks.py          # Script principal Python
3 |-- checks/
4 |   |-- checks_kyc.yml          # Regles DQ colonnes KYC
5 |   |-- checks_csp.yml          # Regles DQ colonnes CSP
6 |   +-- checks_pipeline.yml     # Regles fraicheur & pipeline
7 +-- utils/
8     +-- hive_writer.py          # Module d'écriture resultats dans
      Hive
```

4.2.2 Fichier de configuration YAML — Règles KYC (checks_kyc.yml)

```
1 # checks_kyc.yml
2 # Regles Data Quality - Domaine KYC
3 # Source : raw_nova_referentiel_tiers.ccm_npdescription (zone RAW)
4 # Zone RAW = copie exacte d'Oracle NOVA, aucune transformation
   appliquée
5
6 checks for ccm_npdescription
7
8 # --- Completude ---
9 - missing_count(name) = 0
10     name "KYC-01 | Completude | Nom"
11     fail error
12     attributes
13         criticite CRITIQUE
14         domaine KYC
15         dimension completeness
16
17 - missing_count(first_name) = 0
18     name "KYC-02 | Completude | Prenom"
19     fail error
20     attributes
21         criticite CRITIQUE
22         domaine KYC
```

```

23     dimension completeness
24
25 - missing_count(birthdate) = 0
26   name "KYC-03 | Completude | Date de naissance"
27   fail error
28   attributes
29     criticite CRITIQUE
30     domaine KYC
31     dimension completeness
32
33 - missing_count(nationality_country) = 0
34   name "KYC-04 | Completude | Nationalite"
35   fail warn
36   attributes
37     criticite HAUTE
38     domaine KYC
39     dimension completeness
40
41 # --- Validite ---
42 - invalid_count(civility) = 0
43   valid values ["M.", "MME", "MLLE"]
44   name "KYC-05 | Validite | Civilite hors referentiel"
45   fail warn
46   attributes
47     criticite HAUTE
48     domaine KYC
49     dimension validity
50
51 - invalid_count(gender) = 0
52   valid values ["M", "F"]
53   name "KYC-06 | Validite | Genre hors referentiel"
54   fail warn
55   attributes
56     criticite HAUTE
57     domaine KYC
58     dimension validity
59
60 # --- Fraicheur ---
61 - freshness(batch_datetime_eaac4) < 26h
62   name "KYC-07 | Fraicheur | Retard batch"
63   fail error
64   attributes
65     criticite HAUTE
66     domaine PIPELINE
67     dimension freshness

```

4.2.3 Fichier de configuration YAML — Règles CSP (checks_csp.yml)

```

1 # checks_csp.yml
2 # Regles Data Quality - Domaine CSP
3 # Source : raw_nova_referentieltiers.ccm_job (zone RAW)

```

```

4 # Zone RAW = copie exacte d'Oracle NOVA, aucune transformation
   appliquee
5
6 checks for ccm_job
7
8 # --- Completude triple CSP ---
9 - missing_percent(professional_sector) < 5
10     name "CSP-01 | Completude | Secteur professionnel"
11     fail warn
12     attributes
13         criticite HAUTE
14         domaine CSP
15         dimension completeness
16
17 - missing_percent(occupation) < 5
18     name "CSP-02 | Completude | Code profession"
19     fail warn
20     attributes
21         criticite HAUTE
22         domaine CSP
23         dimension completeness
24
25 - missing_percent(socioprofessional_category) < 5
26     name "CSP-03 | Completude | Code CSP"
27     fail warn
28     attributes
29         criticite HAUTE
30         domaine CSP
31         dimension completeness
32
33 # --- Validdite ---
34 - invalid_count(professional_situation) = 0
35     valid values ["SAL", "FON", "TRI", "AUT", "EMP", "NDE"]
36     name "CSP-04 | Validdite | Situation professionnelle"
37     fail warn
38     attributes
39         criticite HAUTE
40         domaine CSP
41         dimension validity
42
43 # --- Conformite PPE ---
44 - missing_count(politically_exposed) = 0
45     name "CSP-05 | Completude | PPE (Personne Politiquement
46     Exposee)"
47     fail error
48     attributes
49         criticite CRITIQUE
50         domaine CSP
51         dimension completeness

```

4.2.4 Script Python principal (dq_tiers_checks.py)

```

1 # dq_tiers_checks.py
2 # Pipeline de Data Quality - Referentiel Tiers PP Clients - Zone
  RAW
3 # CIH Bank - Data Office
4 # Auteur : Saad BENDAHOU
5 # Declenchement : post-ingestion DAG dailyow22 (22h00)
6
7 import argparse
8 import datetime
9 from datetime import date
10 from pyspark.sql import SparkSession
11 from soda.scan import Scan
12
13 def run_dq_checks(run_date: str):
14     """
15     Execute les controles DQ Soda Core sur les tables Hive
16     du domaine Tiers et persiste les resultats.
17     """
18     # --- 1. Initialisation de la SparkSession ---
19     spark = SparkSession.builder \
20         .appName("DQ_Tiers_Checks") \
21         .enableHiveSupport() \
22         .getOrCreate()
23
24     # --- 2. Initialisation du scan Soda Core ---
25     scan = Scan()
26     scan.set_data_source_name("hive_raw")
27     scan.add_spark_session(spark, "hive_raw")
28
29     # --- 3. Chargement des fichiers de regles YAML ---
30     scan.add_sodacl_yaml_file("checks/checks_kyc.yml")
31     scan.add_sodacl_yaml_file("checks/checks_csp.yml")
32     scan.add_sodacl_yaml_file("checks/checks_pipeline.yml")
33
34     # --- 4. Execution des checks ---
35     scan.execute()
36
37     # --- 5. Collecte et persistance des resultats de faits ---
38     run_timestamp = datetime.datetime.now()
39     results_data = []
40
41     for check in scan.get_checks_self():
42         passed = check.outcome == "pass"
43         score = 100.0 if passed else (50.0 if check.outcome == "
warn" else 0.0)
44
45         # Recupere le total des lignes et le nombre d'anomalies
46         total_rows = spark.table(f"raw_nova_referentieltiers.{check
.dataset_name}").count()
47         invalid_rows = int(check.check_value or 0) if not passed
else 0
48         valid_rows = total_rows - invalid_rows

```

```

49     percentage = (valid_rows / total_rows) * 100.0 if
total_rows > 0 else 100.0
50
51     results_data.append((
52         run_timestamp,
53         "dailyow22",
54         check.dataset_name,
55         check.column_name or "TRANSVERSE",
56         check.name,
57         check.attributes.get('dimension', 'completeness'),
58         check.attributes.get('criticite', 'HAUTE'),
59         check.attributes.get('domaine', 'Tiers'),
60         total_rows,
61         valid_rows,
62         invalid_rows,
63         percentage,
64         passed,
65         score,
66         run_date
67     ))
68
69     if results_data:
70         df = spark.createDataFrame(results_data, [
71             "run_timestamp", "dag_name", "table_name", "column_name",
72             "check_name",
73             "dimension", "criticite", "domaine", "total_rows", "
valid_rows",
74             "invalid_rows", "percentage", "passed", "score_dq", "
run_date"
75         ])
76         df.write.mode("append").partitionBy("run_date").saveAsTable
("dq_results.dq_tiers_results_fact")
77
78     spark.stop()
79     print(f"[DQ Tiers] Controles termines pour la date {run_date}")
80
81 if __name__ == "__main__":
82     parser = argparse.ArgumentParser()
83     parser.add_argument("--run_date", type=str, default=str(date.
today()))
84     args = parser.parse_args()
85     run_dq_checks(args.run_date)

```

4.3 Création des tables Hive de persistance

Les tables de persistance DQ sont créées dans la base Hive dq_results. Le script HiveQL de création est le suivant :

```

1  -- Creation de la base dediee DQ
2  CREATE DATABASE IF NOT EXISTS dq_results
3  COMMENT 'Base de resultats Data Quality - CIH Bank Data Office';
4

```

```

5  -- Table de faits unique pour la Data Quality
6  CREATE TABLE IF NOT EXISTS dq_results.dq_tiers_results_fact (
7      run_timestamp    TIMESTAMP,
8      dag_name         STRING,
9      table_name       STRING,
10     column_name      STRING,
11     check_name       STRING,
12     dimension        STRING,
13     criticite        STRING,
14     domaine          STRING,
15     total_rows       BIGINT,
16     valid_rows       BIGINT,
17     invalid_rows     BIGINT,
18     percentage       DOUBLE,
19     passed           BOOLEAN,
20     score_dq         DOUBLE
21 )
22 PARTITIONED BY (run_date DATE)
23 STORED AS ORC
24 TBLPROPERTIES ('orc.compress'='SNAPPY');

```

4.4 Intégration dans le DAG Airflow dailyow22

4.4.1 Ajout de la tâche DQ

La tâche de Data Quality est ajoutée au DAG dailyow22 en tant qu'opérateur Python, déclenchée après la fin de toutes les tâches d'ingestion Sqoop des tables CCM_*. Le TriggerRule.ALL_DONE garantit que les contrôles DQ s'exécutent même si certaines tâches d'ingestion ont échoué partiellement.

```

1  # Extrait du DAG dailyow22 (Airflow)
2  from airflow.operators.python import PythonOperator
3  from airflow.utils.trigger_rule import TriggerRule
4  import subprocess
5
6  def run_dq_tiers(**context):
7      run_date = context['ds'] # Format YYYY-MM-DD
8      result = subprocess.run(
9          [
10             'python',
11             '/opt/airflow/scripts/dq/dq_tiers_checks.py',
12             '--run_date', run_date
13         ],
14         capture_output=True,
15         text=True
16     )
17     if result.returncode != 0:
18         raise Exception(f"DQ Tiers failed:\n{result.stderr}")
19
20  dq_tiers_task = PythonOperator(
21     task_id='run_dq_tiers_checks',

```

```

22     python_callable=run_dq_tiers,
23     trigger_rule=TriggerRule.ALL_DONE,
24     dag=dag
25 )
26
27 # Positionnement dans le graphe : apres toutes les ingestions CCM_*
28 [ccm_party_task, ccm_npdescription_task, ccm_job_task,
29  ccm_portfolio_task, ccm_context_task] >> dq_tiers_task

```

4.4.2 Planification et monitoring

Le DAG `dailyow22` s'exécute quotidiennement à **22h00**. La tâche DQ `run_dq_tiers_checks` se déclenche donc automatiquement chaque nuit après l'ingestion, sans intervention manuelle. Les logs Airflow permettent de suivre l'exécution et de détecter toute anomalie dans le pipeline DQ lui-même.

4.5 Création des vues Dremio

Trois vues SQL sont créées dans Dremio, dans le nouveau schéma `dq-tiers`, pointant vers les tables Hive de la base `dq_results`.

4.5.1 Vue `v.dq_scores_history`

```

1  -- Vue Dremio : historique des scores DQ
2  CREATE OR REPLACE VIEW "dq-tiers"."v.dq_scores_history" AS
3  SELECT
4      run_date,
5      run_timestamp,
6      dag_name,
7      table_name,
8      column_name,
9      check_name,
10     dimension,
11     criticite,
12     domaine,
13     total_rows,
14     valid_rows,
15     invalid_rows,
16     percentage,
17     passed,
18     score_dq
19 FROM hive.dq_results.dq_tiers_results_fact
20 ORDER BY run_date DESC, criticite ASC;

```

4.5.2 Vue `v.dq_kpis_summary`

```

1  -- Vue Dremio : KPIs agregés par domaine et dimension
2  CREATE OR REPLACE VIEW "dq-tiers"."v.dq_kpis_summary" AS

```

```

3 SELECT
4     run_date,
5     domaine,
6     dimension,
7     criticite,
8     COUNT(*)                AS nb_checks,
9     SUM(CASE WHEN passed THEN 1
10          ELSE 0 END)        AS nb_checks_ok,
11     AVG(score_dq)           AS score_moyen,
12     MIN(score_dq)           AS score_min
13 FROM hive.dq_results.dq_tiers_results_fact
14 GROUP BY run_date, domaine, dimension, criticite
15 ORDER BY run_date DESC, domaine, criticite;

```

4.5.3 Vue v.dq_anomalies_summary

```

1  -- Vue Dremio : regles en anomalie (filtree sur passed = false)
2  CREATE OR REPLACE VIEW "dq-tiers"."v.dq_anomalies_summary" AS
3  SELECT
4      run_date,
5      table_name,
6      column_name,
7      check_name,
8      total_rows,
9      valid_rows,
10     invalid_rows,
11     percentage,
12     criticite,
13     score_dq
14 FROM hive.dq_results.dq_tiers_results_fact
15 WHERE passed = FALSE
16 ORDER BY run_date DESC,
17          CASE criticite
18              WHEN 'CRITIQUE' THEN 1
19              WHEN 'HAUTE'    THEN 2
20              WHEN 'MOYENNE'  THEN 3
21              ELSE 4
22          END;

```

4.6 Réalisation du dashboard Tableau

4.6.1 Connexion Tableau à Dremio

Le dashboard Tableau est connecté à Dremio via le connecteur ODBC/JDBC Dremio. Les trois vues du schéma dq-tiers sont importées comme sources de données dans Tableau : v.dq_scores_history, v.dq_kpis_summary et v.dq_anomalies_summary.

4.6.2 Onglet 1 — Vue Exécutive

La figure 12 présente l’onglet « Vue Exécutive » du dashboard.

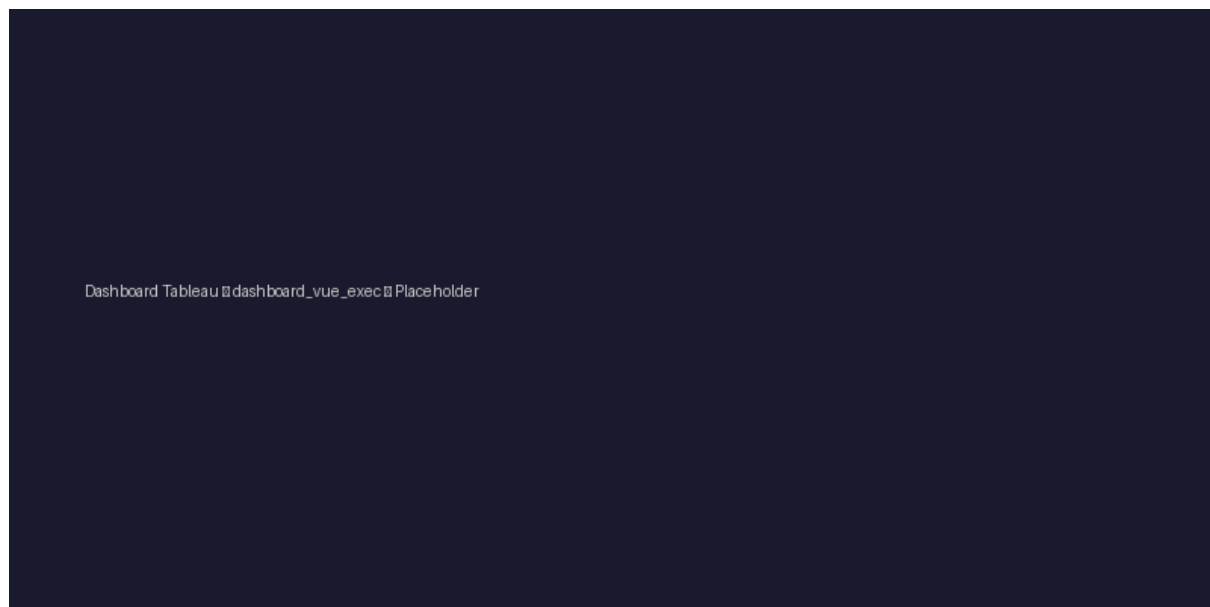


FIGURE 12 – Dashboard Tableau — Vue Exécutive : scores DQ globaux KYC et CSP

Cet onglet affiche :

- Le score DQ global du jour pour le domaine KYC et le domaine CSP (indicateurs chiffrés 0–100)
- La courbe d’évolution des scores sur les 30 derniers jours
- Le nombre d’alertes critiques et hautes en cours
- La date et l’heure de la dernière exécution du batch (`dailyow22`)

4.6.3 Onglet 2 — Détail KYC

La figure 13 présente l’onglet « Détail KYC ».

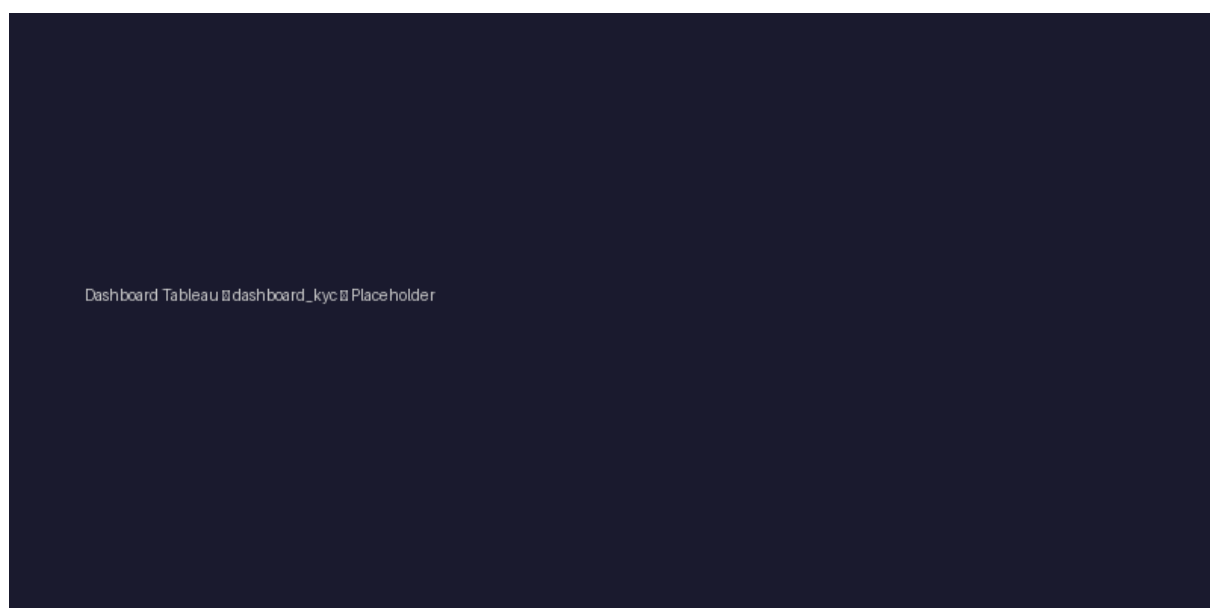


FIGURE 13 – Dashboard Tableau — Détail KYC : score par colonne et dimension DQ

4.6.4 Onglet 3 — Détail CSP

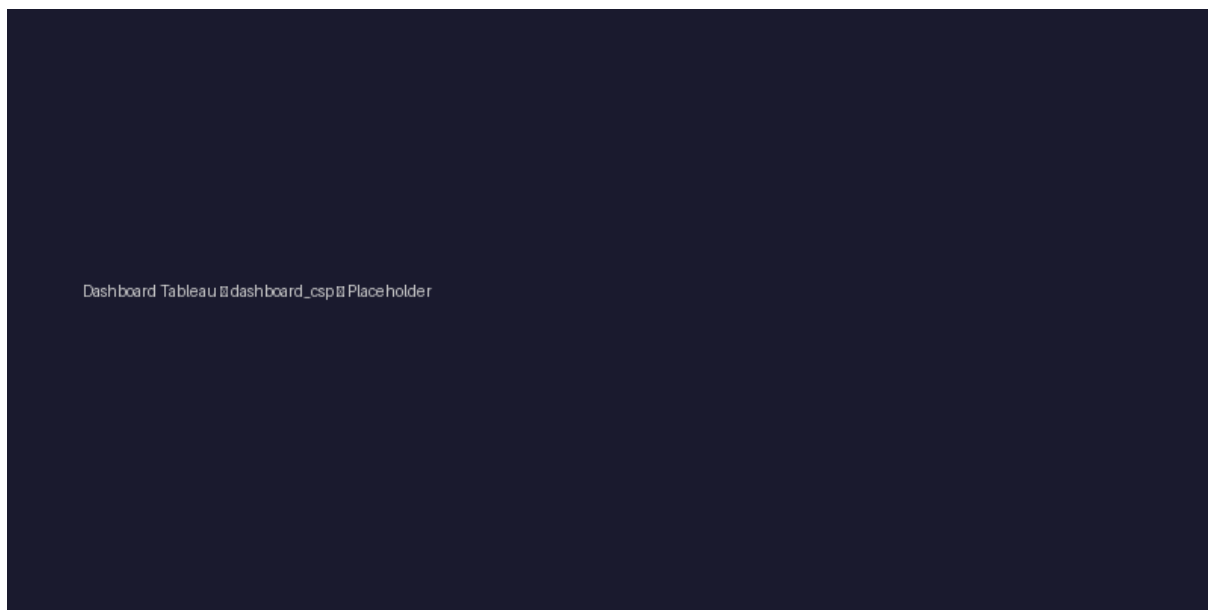


FIGURE 14 – Dashboard Tableau — Détail CSP : taux de nullité et valeurs parasites

4.6.5 Onglet 4 — Anomalies et Remédiation

Le quatrième onglet présente le résumé des règles de qualité en échec (triée par criticité décroissante). Elle permet aux Data Stewards de cibler directement les colonnes et les règles de gestion présentant des taux d'anomalies élevés pour engager des actions de correction collectives dans NOVA. Les colonnes affichées sont : `table_name`, `column_name`, `check_name`, `total_rows`, `invalid_rows`, `percentage`, `criticite`, `run_date`.

Conclusion

Ce chapitre a présenté l'implémentation concrète des différents composants du pipeline de Data Quality Tiers : le script Soda Core avec ses fichiers de configuration YAML, les tables Hive de persistance, la tâche Airflow post-ingestion, les vues Dremio d'exposition et le dashboard Tableau de visualisation. L'ensemble de ces composants forme un pipeline automatisé, opérationnel chaque nuit à 22h, qui permet de mesurer, historiser et restituer la qualité des données KYC et CSP du Référentiel Tiers. Le chapitre suivant présente les résultats obtenus lors des premières exécutions. Chapitre 5 : Résultats et analyse

Conclusion générale